For the application in listing 3.3, we didn't add any middleware, but we defined a single endpoint using a call to `MapGet`:

```
app.MapGet("/", () => "Hello World!");
```

You use the `MapGet` function to define how to handle a request that uses the `GET` *HTTP verb*. There are other `Map*` functions for other HTTP verbs, such as `MapPost`.

DEFINITION Every HTTP request includes a *verb* that indicates the type of the request. When you're browsing a website, the default verb is `GET`, which *fetches* a resource from the server so you can view it. The second-most-common verb is `POST`, which is used to *send* data to the server, such as when you're completing a form.

The first argument passed to `MapGet` defines which URL path to respond to, and the second argument defines *how* to generate the response as a delegate that returns a `string`. In this simple case, the arguments say "When a request is made to the path `/` using the `GET` *HTTP verb*, respond with the plain-text value `Hello World!`".

DEFINITION A *path* is the remainder of the request URL after the domain has been removed. For a request to www.example.org/accout/manage, the path is `/account/manage`.

While you're configuring the `WebApplication` and `WebApplicationBuilder` the application isn't handling HTTP requests. Only after the call to `Run()` does the HTTP server start listening for requests. At this point, your application is fully operational and can respond to its first request from a remote browser.